JAVA LAB RECORD

Java Collection Framework: Important Methods by Class

1. Aim

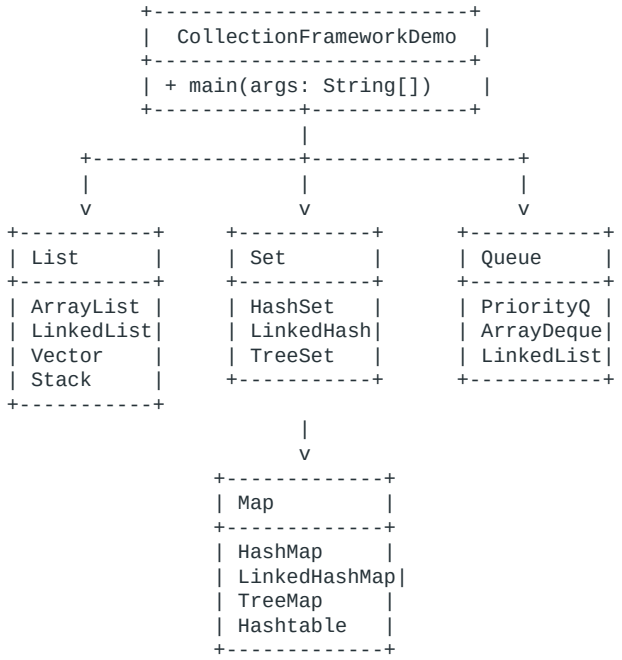
To demonstrate important methods of Java Collection Framework classes such as ArrayList, LinkedList, Vector, Stack, Set implementations, Queue/Deque implementations, Map implementations, and Hashtable.

2. Description

The Java Collection Framework provides interfaces and classes to store and manipulate groups of objects. Lists maintain sequence and can allow duplicates; sets store unique elements; queues process elements according to queue/priority rules; and maps store key-value pairs. This program demonstrates commonly used methods such as add(), get(), set(), remove(), contains(), size(), push(), pop(), offer(), poll(), peek(), put(), and get().

Important: HashSet and HashMap do not guarantee iteration order. PriorityQueue displays elements in heap order, so its printed representation is not necessarily fully sorted. Vector and Hashtable are synchronized legacy classes.

3. UML Diagram



UML summary: The main() method creates objects from different collection classes and calls their important methods to demonstrate how each class behaves.

4. Source Code with Comments

Save the file as **CollectionFrameworkDemo.java**. The public class name must match the file name.

```
// Java Collection Framework: Demonstration of important methods
// Save this file as CollectionFrameworkDemo.java

import java.util.*;

public class CollectionFrameworkDemo {
    public static void main(String[] args) {

        // 1. ArrayList: allows duplicates and maintains insertion order
        ArrayList<Integer> list = new ArrayList<>();
        list.add(30);
        list.add(10);
        list.add(20);
        list.add(10);
        list.add(1, 15);                // Insert at index 1
        System.out.println("ArrayList: " + list);
        System.out.println("ArrayList get(0): " + list.get(0));
        list.set(0, 5);                 // Replace element at index 0
        System.out.println("ArrayList after set: " + list);
        System.out.println("ArrayList contains 20: " + list.contains(20));
        System.out.println("ArrayList size: " + list.size());
        System.out.println("ArrayList indexOf(10): " + list.indexOf(10));
        list.remove(Integer.valueOf(10)); // Remove first matching value
        System.out.println("ArrayList after remove: " + list);

        // 2. LinkedList: supports operations at both ends
        LinkedList<String> linkedList = new LinkedList<>();
        linkedList.add("B");
        linkedList.addFirst("A");
        linkedList.addLast("C");
        System.out.println("\\nLinkedList: " + linkedList);
        System.out.println("LinkedList getFirst(): " + linkedList.getFirst());
        System.out.println("LinkedList getLast(): " + linkedList.getLast());
        System.out.println("LinkedList poll(): " + linkedList.poll());
        System.out.println("LinkedList after poll: " + linkedList);

        // 3. Vector: synchronized resizable array
        Vector<String> vector = new Vector<>();
        vector.add("Java");
        vector.addElement("Python");
        System.out.println("\\nVector: " + vector);
        System.out.println("Vector size: " + vector.size());
```

4. Source Code with Comments (continued)

```
        System.out.println("Vector capacity: " + vector.capacity());

        // 4. Stack: follows Last-In-First-Out (LIFO)
        Stack<Integer> stack = new Stack<>();
        stack.push(10);
        stack.push(20);
        stack.push(30);
        System.out.println("\\nStack: " + stack);
        System.out.println("Stack peek(): " + stack.peek());
        System.out.println("Stack pop(): " + stack.pop());
        System.out.println("Stack after pop: " + stack);

        // 5. HashSet: stores unique values; order is not guaranteed
        HashSet<Integer> hashSet = new HashSet<>();
        hashSet.add(30);
        hashSet.add(10);
        hashSet.add(20);
        hashSet.add(10);                // Duplicate is ignored
        System.out.println("\\nHashSet: " + hashSet);
        System.out.println("HashSet contains 20: " + hashSet.contains(20));
        System.out.println("HashSet size: " + hashSet.size());

        // 6. LinkedHashSet: keeps insertion order and removes duplicates
        LinkedHashSet<String> linkedHashSet = new LinkedHashSet<>();
        linkedHashSet.add("Red");
        linkedHashSet.add("Blue");
        linkedHashSet.add("Red");
        System.out.println("\\nLinkedHashSet: " + linkedHashSet);

        // 7. TreeSet: stores unique elements in sorted order
        TreeSet<Integer> treeSet = new TreeSet<>();
        treeSet.add(30);
        treeSet.add(10);
        treeSet.add(20);
        treeSet.add(10);
        System.out.println("\\nTreeSet: " + treeSet);
        System.out.println("TreeSet first(): " + treeSet.first());
        System.out.println("TreeSet higher(20): " + treeSet.higher(20));

        // 8. PriorityQueue: head is the smallest value by default
        PriorityQueue<Integer> priorityQueue = new PriorityQueue<>();
        priorityQueue.offer(30);
```

4. Source Code with Comments (continued)

```
        priorityQueue.offer(10);
        priorityQueue.offer(20);
        System.out.println("\\nPriorityQueue peek(): " + priorityQueue.peek());
        System.out.println("PriorityQueue poll(): " + priorityQueue.poll());
        System.out.println("PriorityQueue after poll: " + priorityQueue);

        // 9. ArrayDeque: can add and remove at both ends
        ArrayDeque<String> deque = new ArrayDeque<>();
        deque.addFirst("B");
        deque.addFirst("A");
        deque.addLast("C");
        System.out.println("\\nArrayDeque: " + deque);
        System.out.println("ArrayDeque peekFirst(): " + deque.peekFirst());
        System.out.println("ArrayDeque pollLast(): " + deque.pollLast());
        System.out.println("ArrayDeque after pollLast: " + deque);

        // 10. HashMap: stores key-value pairs; key order is not guaranteed
        HashMap<Integer, String> hashMap = new HashMap<>();
        hashMap.put(1, "Java");
        hashMap.put(2, "Python");
        hashMap.put(3, "C++");
        System.out.println("\\nHashMap: " + hashMap);
        System.out.println("HashMap get(2): " + hashMap.get(2));
        System.out.println("HashMap containsKey(1): " + hashMap.containsKey(1));
        System.out.println("HashMap keys: " + hashMap.keySet());

        // 11. LinkedHashMap: preserves insertion order of entries
        LinkedHashMap<Integer, String> linkedHashMap = new LinkedHashMap<>();
        linkedHashMap.put(2, "Two");
        linkedHashMap.put(1, "One");
        linkedHashMap.put(3, "Three");
        System.out.println("\\nLinkedHashMap: " + linkedHashMap);

        // 12. TreeMap: sorts entries by key
        TreeMap<Integer, String> treeMap = new TreeMap<>();
        treeMap.put(3, "Three");
        treeMap.put(1, "One");
        treeMap.put(2, "Two");
        System.out.println("\\nTreeMap: " + treeMap);
        System.out.println("TreeMap firstKey(): " + treeMap.firstKey());
        System.out.println("TreeMap higherKey(1): " + treeMap.higherKey(1));

        // 13. Hashtable: synchronized map; does not allow null keys or values
        Hashtable<Integer, String> hashtable = new Hashtable<>();
        hashtable.put(1, "Apple");
        hashtable.put(2, "Mango");
        System.out.println("\\nHashtable: " + hashtable);
        System.out.println("Hashtable get(1): " + hashtable.get(1));
        System.out.println("Hashtable size: " + hashtable.size());

        System.out.println("\\nAll collection classes demonstrated successfully.");
    }
}
```

5. Terminal-Style Execution

Open a terminal in the folder where CollectionFrameworkDemo.java is saved, then run:

```
$ javac CollectionFrameworkDemo.java
$ java CollectionFrameworkDemo
```

6. Output Screenshot (Terminal Style)

```
● Terminal — Java Output
```

```
JAVA COLLECTION FRAMEWORK - IMPORTANT METHODS
-----------------------------------------
ArrayList: [30, 15, 10, 20, 10]
ArrayList get(0): 30
ArrayList after set: [5, 15, 10, 20, 10]
ArrayList contains 20: true
ArrayList size: 5
ArrayList indexOf(10): 2
ArrayList after remove: [5, 15, 20, 10]

LinkedList: [A, B, C]
LinkedList getFirst(): A
LinkedList getLast(): C
LinkedList poll(): A
LinkedList after poll: [B, C]

Vector: [Java, Python]
Vector size: 2
Vector capacity: 10

Stack: [10, 20, 30]
Stack peek(): 30
Stack pop(): 30
Stack after pop: [10, 20]

HashSet: [order may vary]
HashSet contains 20: true
HashSet size: 3

LinkedHashSet: [Red, Blue]

TreeSet: [10, 20, 30]
TreeSet first(): 10
TreeSet higher(20): 30

PriorityQueue peek(): 10
PriorityQueue poll(): 10
PriorityQueue after poll: [20, 30]

ArrayDeque: [A, B, C]
ArrayDeque peekFirst(): A
ArrayDeque pollLast(): C
ArrayDeque after pollLast: [A, B]

HashMap: {1=Java, 2=Python, 3=C++}
HashMap get(2): Python
HashMap containsKey(1): true
HashMap keys: [1, 2, 3]

LinkedHashMap: {2=Two, 1=One, 3=Three}

TreeMap: {1=One, 2=Two, 3=Three}
TreeMap firstKey(): 1
TreeMap higherKey(1): 2

Hashtable: {2=Mango, 1=Apple}
Hashtable get(1): Apple
Hashtable size: 2

All collection classes demonstrated successfully.
```

Note: HashSet and HashMap output order can vary by Java implementation/runtime. The values shown above are illustrative; the program's exact output may display those entries in a different order.

7. Result

Thus, the program demonstrates commonly used methods of major Java Collection Framework classes and shows how lists, sets, queues, deques, and maps store and manipulate data.